МИФИ

Дисциплина

«Эволюционное программирование и генетические алгоритмы»

ЛАБОРАТОРНАЯ РАБОТА № 1

«Многомерная оптимизация сложной функции генетическим алгоритмом»

Функция Растригина · размерность d = 9

Исходное выражение размерности: d = 5 + (22 mod 6)

Функция взята из строки V = 3 методики; d = 9 задана отдельно.

Москва, 2026

Содержание

Содержание 2

1. Основание и уточнение варианта 3

2. Цель работы 3

3. Математическая постановка 3

4. Теоретические основания и метод решения 3

5. Конфигурации исследования 5

6. Программная реализация и проверка 6

7. Экспериментальная часть 7

8. Графики сходимости и сравнения 9

9. Анализ результатов 11

10. Вывод 11

11. Вопросы к защите и ответы 11

12. Использованные материалы и ограничения 13

Приложение А. Полный листинг main.py 14

Приложение Б. Тесты и воспроизводимость 21

Оглавление кликабельное: названия разделов ведут на соответствующие страницы.

1. Основание и уточнение варианта

Согласно методическому комплексу, лабораторная работа № 1 посвящена многомерной непрерывной оптимизации генетическим алгоритмом с вещественным представлением, ограниченной областью поиска и обязательным экспериментальным сравнением. Методика задаёт номер варианта V в диапазоне 1–20 и рассчитывает размерность как d = 5 + (V mod 6).

Для данной переработки получено отдельное указание **d = 5 + (22 mod 6) = 9**, поскольку 22 mod 6 = 4. Число 22 выходит за диапазон допустимых номеров V в методике; поэтому нельзя документально называть его стандартным вариантом V = 22. Как и в предыдущем отчёте, предметом оптимизации оставлена **функция Растригина из строки V = 3**, но она исследуется в девяти измерениях по отдельно заданному значению d. Приведённые результаты относятся именно к такой явно сформулированной постановке, а не к официально определённому по N, Q, Y индивидуальному варианту.

Исходный методический документ: Р. В. Душкин, «Эволюционное программирование и генетические алгоритмы: комплекс лабораторных работ», с. 2–8. Использованы фотографии рукописного конспекта: модель эволюционного поиска, воспроизводимость и бюджет вычислений, обработка ограничений и элитизм, примеры одномерной оптимизации (страницы конспекта 11–18).

2. Цель работы

Освоить базовый генетический алгоритм для непрерывной многомерной оптимизации: вещественное представление, оценивание целевой функции, селекцию, рекомбинацию, мутацию, обработку ограничений и экспериментальный анализ.

3. Математическая постановка

Нужно приближённо найти глобальный минимум функции Растригина в девятимерной ограниченной области. Особь представляет собой вектор вещественных значений; отдельное декодирование не требуется.

$$D = [-5.12, 5.12]^9; \quad x = (x_1, \ldots, x_9) \in D; \quad \min_{x \in D} f(x)$$

$$f(x) = 10d + \Sigma_{i=1..d} \left[x_i^2 - 10 \cos(2\pi x_i) \right], \quad d = 9$$

Для любой координаты t выполнено $t^2 + 10(1 - \cos(2\pi t)) \geq 0$. Следовательно, $f(x) \geq 0$, и минимум равен нулю в $x^* = (0, \ldots, 0)$. Это известный ориентир для оценки точности, а не подсказка алгоритму: в программной реализации точка x^* не используется.

Растригин многомодален: периодический косинус создаёт множество локальных впадин. Простому локальному поиску сложно покинуть выбранную впадину, а равномерная случайная выборка редко оказывается близко к нулю одновременно по всем девяти координатам за ограниченный бюджет.

4. Теоретические основания и метод решения

Согласно приложенным конспектам, эволюционный поиск поддерживает популяцию конкурирующих решений. В цикле выполняются оценивание, отбор, вариация и формирование следующего поколения. Результат стохастичен: единственный удачный запуск не является свидетельством устойчивой эффективности. Поэтому эксперимент использует независимые seed и статистический анализ.

4.1. Генотип, инициализация и фитнес

Генотип и фенотип совпадают: массив из 9 вещественных координат непосредственно задаёт точку пространства D. Начальная популяция содержит μ = 80 особей; все координаты независимо выбираются из U(−5.12, 5.12). Исходное значение f(x) используется напрямую как критерий минимизации. Для селекции не требуется неустойчивое масштабирование в положительную рулеточную фитнес-функцию.

4.2. Отбор родителей

Используется турнирная селекция с размером турнира k = 3. Для каждого родителя случайно выбираются три индекса из действующей популяции; побеждает особь с минимальным f(x). Один и тот же кандидат может участвовать в разных турнирах.

4.3. Вещественный кроссовер BLX-α

С вероятностью p_{cross} = 0.90 для каждого потомка выполняется BLX-α с α = 0.5. Для координаты двух родителей a_i и b_i расширенный интервал строится по длине $I_i = |a_i - b_i|$; из него независимо выбирается потомок.

$$c_i \sim U(\min(a_i, b_i) - \alpha I_i,\ \max(a_i, b_i) + \alpha I_i)$$

Если кроссовер не применяется, потомок до мутации копирует первого родителя. Оператор выбран как непосредственно работающий с вещественными векторами; это также реализованный пункт дополнительного задания (BLX-α).

4.4. Гауссовская мутация

Каждая из девяти координат потомка независимо изменяется с вероятностью p_{mut}. К выбранной координате прибавляется шум ε, распределённый нормально с нулевым математическим ожиданием. Дисперсионный масштаб σ_g заранее уменьшается по экспоненте от 2.50 до 0.03. Это запланированное уменьшение, а не самоадаптация на основе результатов алгоритма.

$$x'_i = x_i + \varepsilon; \quad \varepsilon \sim N(0, \sigma_g^{\,2})$$

$$\sigma_g = 2.50 \times (0.03 / 2.50)^{g/(G-1)}, \quad G = 250$$

4.5. Границы, элитизм, останов

После кроссовера и мутации каждая координата потомка восстанавливается отсечением по [−5.12, 5.12]. Две особи с наименьшими f(x) переносятся в новое поколение без изменений (элитизм). Дополнительно хранится лучшее решение за все поколения best-ever. Остановка осуществляется после G = 250 поколений.

$$\text{Бюджет оценок} = 80 + (250 - 1) \times (80 - 2) = 19\ 502$$

Для равного сравнения случайный поиск оценивает ровно 19 502 независимо выбранные точки, сгруппированные в пакеты того же размера. Элиты не оцениваются повторно, поэтому фактический счётчик функции соответствует указанной формуле.

5. Конфигурации исследования

Параметр	Значение
Функция	Растригина (строка 3 методики); d = 9 по отдельному условию
Область	[−5.12, 5.12] в каждой из 9 координат
Популяция; поколений	80; 250
Бюджет функции	19 502 оценки на один запуск
Селекция	Турнирная, k = 3
Кроссовер	BLX-α, α = 0.5; p_cross = 0.90
Конфигурация A	p_mut = 1/(2d) = 1/18 ≈ 0.055556
Конфигурация B	p_mut = 2/d = 2/9 ≈ 0.222222
Изменяемый фактор	Только вероятность мутации; остальные параметры равны
Мутация; σ	Гауссовская по координатам; от 2.50 до 0.03
Ограничения	clip в допустимую область
Элитизм	2 особи; best-ever сохраняется
Seed	2026100200–2026100219, по 20 независимых запусков

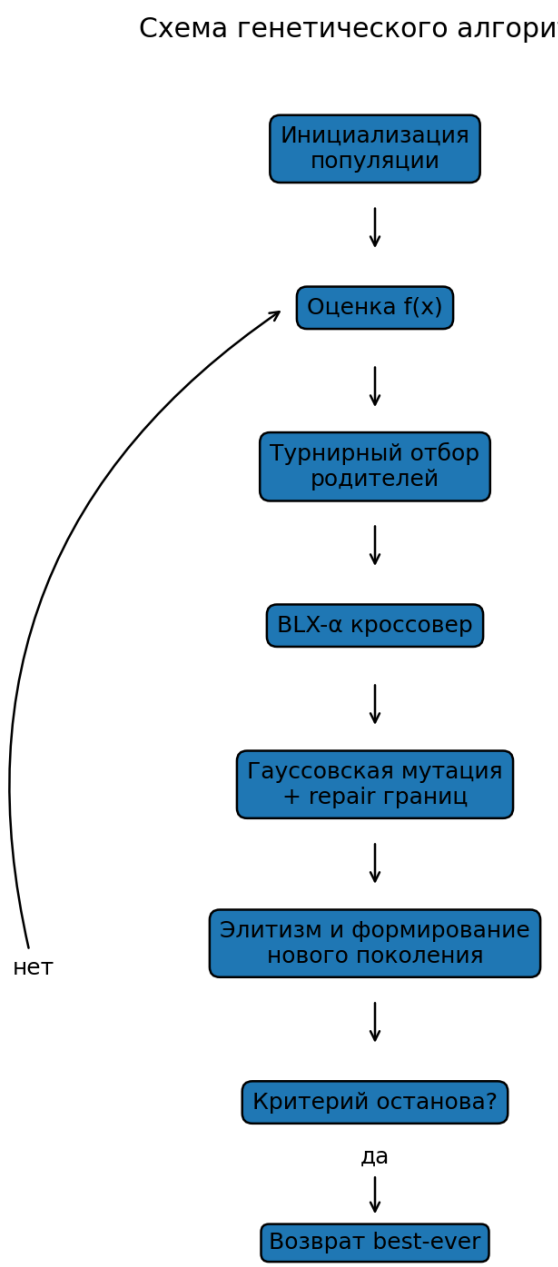


Рисунок 1 — схема реализованного генетического алгоритма. Возврат к оцениванию показан, пока критерий остановки не выполнен.

6. Программная реализация и проверка

Программа написана на Python. NumPy служит для вычислений, массивов и генерации псевдослучайных чисел; Matplotlib — исключительно для графиков. Готовый оптимизатор не используется. Турнирная селекция, BLX-α-кроссовер, мутация, обработка границ, элитизм, основной цикл поиска и сравниваемый случайный baseline реализованы самостоятельно.

Все настройки вынесены в структуру GAConfig. Запуски принимают seed, алгоритм собирает траекторию лучшего известного значения по поколениям, фиксирует число обращений к f(x) и сохраняет точный вектор итогового решения. Исходный код доступен полностью в приложении А и в комплекте файлов.

Запуск эксперимента: `python main.py --output-dir results --runs 20 --seed-base 2026100200`

Автотесты: `python -m unittest -v test_lab1.py`

Проверены: f(0) = 0, допустимость после clip, воспроизводимость одинакового seed, девятимерность возвращённого решения и точность счётчика вычислений. Выполнено 5 из 5 автотестов. Эксперимент заново проведён для d = 9; прежние восьмимерные результаты не переносились.

7. Экспериментальная часть

7.1. План и честность сравнения

• Выполнить 20 независимых запусков конфигурации A и 20 запусков конфигурации B при тех же seed. Влияет только p_mut.

• Выполнить 20 запусков случайного поиска с тем же числом обращений к f(x), что у каждой конфигурации ГА.

• Сохранить итоговые значения и 9 координат всех решений в runs.csv; все траектории best-ever — в trajectories.csv.

• Рассчитать лучшее, среднее, медиану, выборочное стандартное отклонение и худшее значение на каждой группе из 20 запусков.

7.2. Итоговая статистика 20 запусков каждой конфигурации

Метод	Лучшее	Среднее	Медиана	Ст. откл.	Худшее
ГА A: p_mut = 1/18	1.4623e-11	1.84067	1.98992	1.30228	4.97480
ГА B: p_mut = 2/9	1.02661	5.90805	6.00073	2.49328	10.96253
Случайный поиск	34.4499	51.99001	52.14786	7.41526	63.26653

7.3. Результаты каждого независимого запуска

Seed	ГА A: f_min	ГА B: f_min	Случ. поиск: f_min
2026100200	0.994959	6.98997	53.0441
2026100201	1.98992	5.99396	35.6638
2026100202	1.98992	10.9625	57.1589
2026100203	1.98992	7.99336	48.9081
2026100204	1.74779e-10	6.00749	52.8201
2026100205	1.98992	4.0001	53.5584
2026100206	2.27122e-09	9.97152	61.2486
2026100207	1.98992	2.03489	58.808
2026100208	2.98488	7.00224	51.4756
2026100209	3.90024e-09	5.02165	63.2665
2026100210	2.98488	8.99396	58.2912
2026100211	1.4623e-11	4.02058	45.9435
2026100212	4.9748	6.03322	59.7102
2026100213	0.994959	5.01919	48.8797
2026100214	2.98488	7.98099	55.4106
2026100215	1.98992	4.00232	50.917
2026100216	2.98488	4.03677	49.7984
2026100217	2.98488	6.0161	49.5181
2026100218	1.98992	1.02661	34.4499
2026100219	0.994959	5.05355	50.9296

7.4. Лучший найденный вектор и проверка

Лучший результат основной конфигурации получен при seed = 2026100211: f(x_best) = 1.46229695019e-11. Это экспериментальный результат, а не гарантия для других запусков.

Координата	Значение
x1	+7.978659390523e-08
x2	+1.283896607692e-08
x3	-3.145175675599e-09
x4	-5.494956279112e-08
x5	-4.383314893251e-09
x6	+5.385156521672e-08

Координата	Значение
x7	-1.751932294516e-08
x8	-2.172483516275e-08
x9	+2.457113167476e-07

Теоретический минимум равен 0 при x* = (0,...,0). Все девять компонент найденного лучшего вектора близки к нулю; точность значения f(x_best) относится только к показанному запуску. Для корректной оценки устойчивости важны также среднее и медиана по серии.

8. Графики сходимости и сравнения

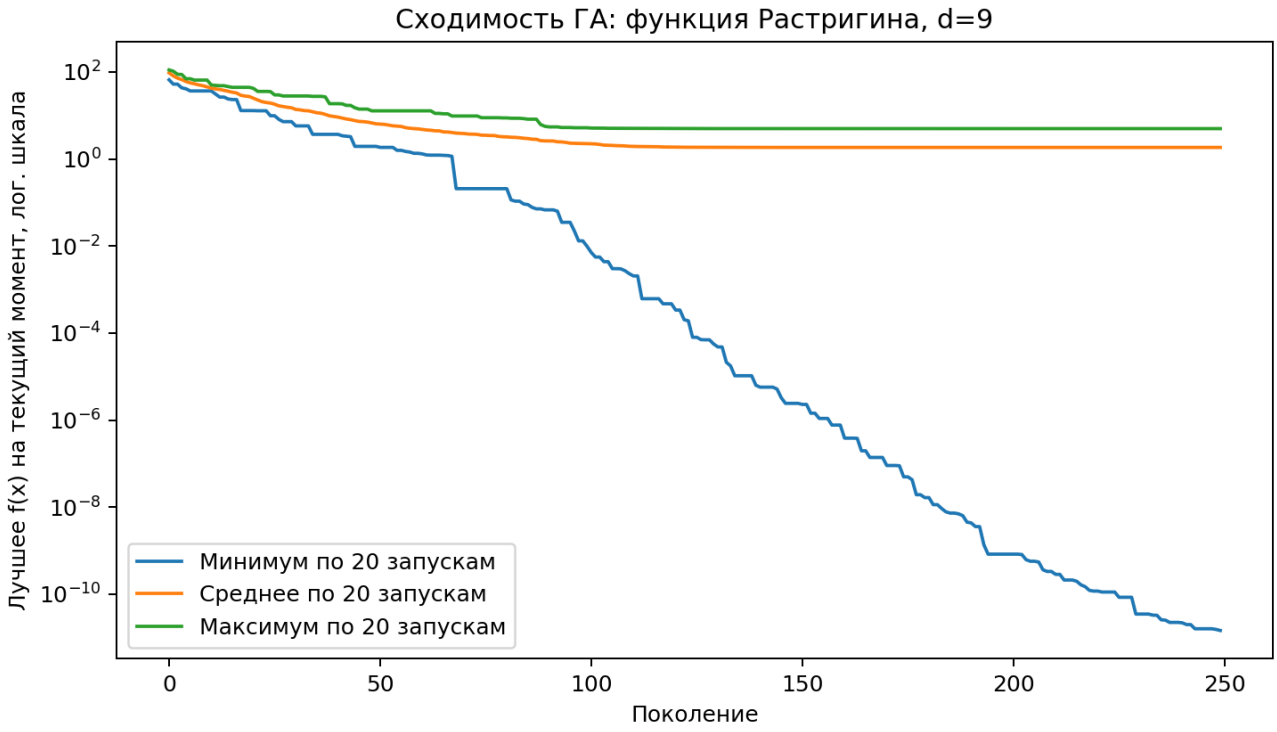


Рисунок 2 — минимум, среднее и максимум лучшего к текущему поколению значения по 20 запускам ГА А (логарифмическая шкала).

Различие между кривыми отражает чувствительность стохастического процесса к seed: лучшие траектории приближаются к нулю, но часть запусков заканчивается около локальных минимумов. Логарифмический масштаб позволяет увидеть как крупные начальные, так и очень малые итоговые значения.

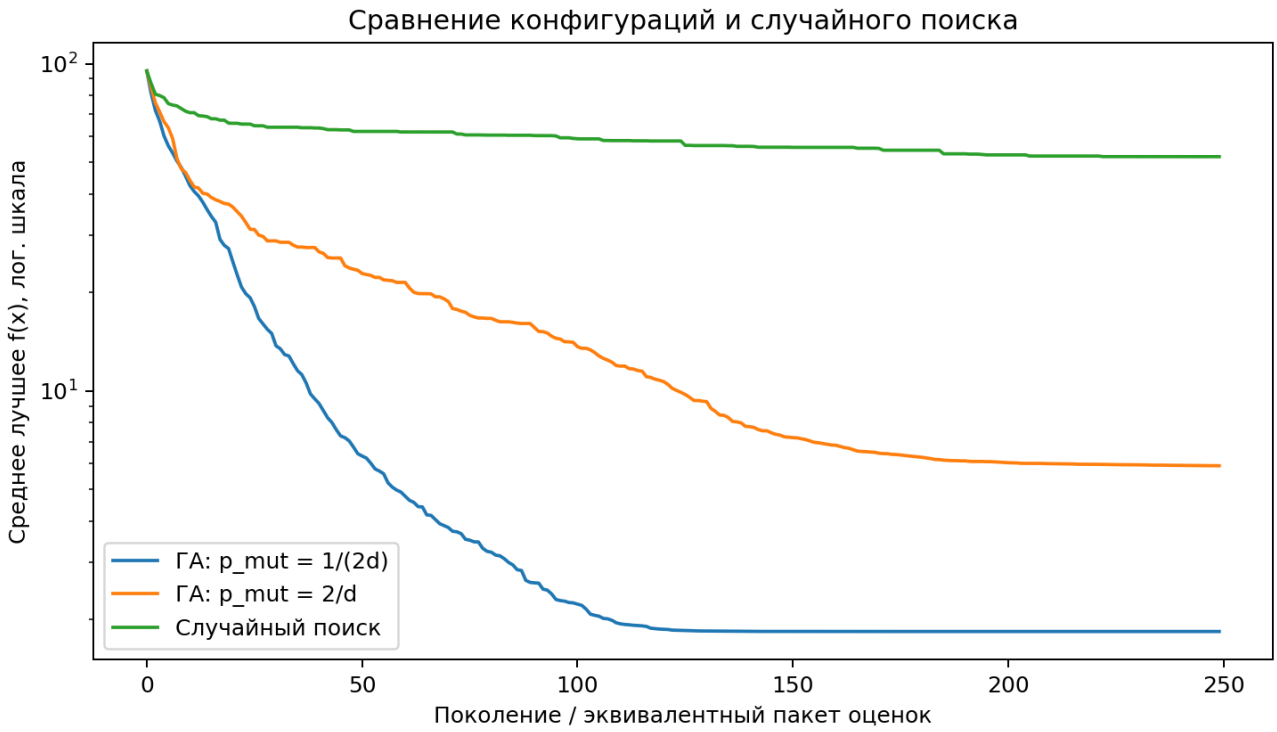


Рисунок 3 — средние траектории конфигураций A, B и случайного поиска при одинаковом числе вычислений функции.

На каждом шаге сравнивается одинаковое накопленное число вычислений f(x): 80 стартовых оценок и по 78 последующих. Отличие A и B — только вероятность изменения каждой координаты.

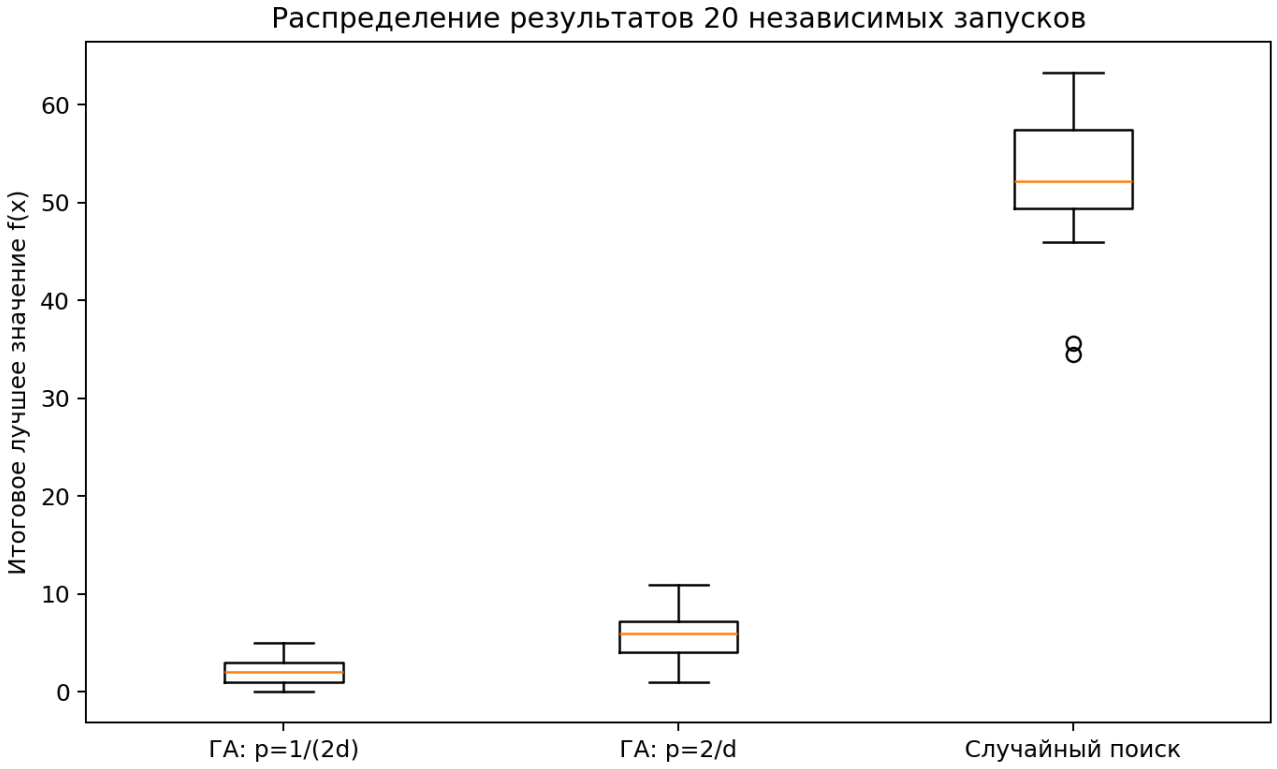


Рисунок 4 — распределение итогового лучшего значения на 20 запусках каждого метода.

9. Анализ результатов

Основная конфигурация A достигла окрестности глобального минимума $f(x) < 10^{-6}$ в 4 из 20 запусков (20%). При этом медиана её итогов равна 1.989918: удачные запуски не характеризуют типичный результат. Траектории, оканчивающиеся на плато вблизи характерных положительных уровней, согласуются с попаданием в локальные минимумы Растригина.

Среднее значение для A составило 1.840674, для B — 5.908051; в данной фиксированной экспериментальной схеме увеличение p_mut с 1/18 до 2/9 сопровождалось ухудшением среднего результата примерно в 3.21 раза. Этот вывод относится к испытанным параметрам и бюджету; оптимальную вероятность мутации вообще из него вывести нельзя.

Случайный поиск получил среднее 51.990015; это примерно в 28.25 раза выше (хуже при минимизации), чем среднее у A. Оба метода использовали 19 502 обращения к функции и по 20 seed; преимущество в данной серии связано с использованием ГА информации об уже исследованных точках. Универсального доказательства превосходства ГА отсюда не следует.

Существенная неопределённость остаётся: измерялись итоговые значения и динамика лучшего решения, но не вычислялась отдельная метрика разнообразия всей популяции. Поэтому плато служат косвенным свидетельством возможной преждевременной сходимости, а не исчерпывающей диагностикой механизма.

10. Вывод

Для девятимерной функции Растригина самостоятельно реализован вещественный генетический алгоритм с турнирной селекцией, BLX-α, гауссовской мутацией, отсечением по границам, элитизмом и хранением лучшего решения за все поколения. Проведены 60 воспроизводимых запусков, собраны машиночитаемые результаты, таблицы и графики; обе конфигурации ГА и случайный поиск сравнивались при одинаковом бюджете 19 502 оценки.

В основной конфигурации лучший результат составил 1.4623e-11; средний — 1.8407 против 51.9900 у случайного поиска. Изменение только вероятности мутации существенно влияло на результат. При увеличении вычислительного бюджета целесообразно отдельно исследовать рестарты, различные масштабы мутации, поддержку разнообразия и более широкий набор функций.

11. Вопросы к защите и ответы

1. Каково пространство решений и почему выбранное представление его корректно описывает?

Область $D = [-5.12, 5.12]$ в девятой степени. Особь — вектор из девяти вещественных координат. Такое представление непосредственно задаёт любую точку допустимой области.

2. Чем различаются генотип и фенотип в вашей работе?

Генотип хранится как числовой массив длины 9, фенотип — девятимерный вектор параметров, подставляемый в Растригина. Декодирование тождественно.

3. Как определяется фитнес-функция и не искажает ли она исходную цель задачи?

В алгоритме используется исходная функция $f(x)$. Поскольку решается задача минимизации, турнир выбирает кандидата с минимальным $f(x)$. Преобразование цели не требуется.

4. Какие решения считаются допустимыми? Как обрабатываются нарушения ограничений?

Допустимы только векторы с каждой координатой в интервале [−5.12, 5.12]. Любой выход за границы после рекомбинации и мутации устраняется clip до вычисления цели.

5. Почему выбран именно этот оператор кроссовера и эта мутация?

BLX-α соответствует вещественному генотипу, способен интерполировать и немного экстраполировать координаты родителей. Гауссовская мутация обеспечивает непрерывную вариацию с контролируемым масштабом.

6. Может ли ваш кроссовер создать недопустимое решение? Что происходит в таком случае?

Да, расширенный интервал BLX-α иногда выходит за допустимые границы, как и мутация. Потомок до оценивания ремонтируется отсечением координат.

7. Как определяется селективное давление в вашей реализации?

Основной механизм — турнир размера k = 3. Более крупный турнир увеличил бы шанс выбора лучших родителей. Элитизм дополнительно гарантирует сохранение двух лучших особей.

8. Почему нельзя делать вывод об эффективности по одному запуску?

Результат зависит от случайной инициализации, турниров, скрещивания и мутаций. Для статистически осмысленного сравнения выполнены 20 независимых запусков на каждую конфигурацию.

9. Какие seed использовались в эксперименте и можно ли воспроизвести лучший результат?

Использованы seed 2026100200–2026100219. Лучший результат A получен при seed 2026100211. Повторный запуск с указанным seed-base и тем же окружением восстанавливает серию.

10. Какой вычислительный бюджет использовался и почему сравнение алгоритмов является честным?

На запуск приходится ровно 19 502 оценки: 80 исходных + 249 × 78 новых потомков/случайных точек. Конфигурации ГА отличаются только p_mut.

11. Как проявляется преждевременная сходимость и как вы её диагностировали?

Косвенные признаки — длительные плато best-ever и скопление итоговых значений выше нуля. Прямая метрика разнообразия популяции в данной реализации не записывается.

12. Почему ваша задача не сводится тривиально к известному точному методу или почему эволюционный подход всё же оправдан?

Для самой тестовой функции глобальный минимум известен аналитически, поэтому ГА применяется здесь не по необходимости, а как учебный объект: многомодальность позволяет измерить способность поиска исследовать локальные впадины.

13. Как изменится алгоритм при росте размерности, числа объектов или числа ограничений?

При увеличении d пространство поиска усложняется; при прежнем бюджете попадание вблизи всех координат оптимума становится труднее. Имеет смысл увеличить бюджет, проверить разнообразие, мутацию и специальные методы ограничений.

14. Какие улучшения вы предложили бы при увеличении доступного вычислительного бюджета?

Рестарты после стагнации, управление масштабом и вероятностью мутации по измеренному разнообразию, локальное уточнение и сравнение с другими эволюционными методами.

15. Какие результаты вашей работы являются экспериментальными, а какие имеют строгую математическую гарантию?

Из определения функции строго следует $f(x) \geq 0$ и $f(0)=0$. Найденное приближение, распределение результатов, скорость сходимости и сравнение методов подтверждены только выполненной экспериментальной серией, а не доказаны для всех запусков.

12. Использованные материалы и ограничения

• Р. В. Душкин. «Эволюционное программирование и генетические алгоритмы. Комплекс лабораторных работ», с. 2–8: общие требования, задача №1, таблица функций, план обязательных экспериментов. Ориентировочные вопросы к защите — с. 23–24.

• 18 приложенных фотографий конспектов: схема стохастического эволюционного алгоритма, ограниченный бюджет, элитизм, обработка границ, вариация, интерпретация однопараметрического примера; прежде всего страницы рукописного конспекта 11–18.

• Внешние источники не использовались. Неоднозначность исходных данных: выражение $d = 5 + (22 \bmod 6)$ задаёт девятимерность отдельно, но номер 22 не является допустимым V в методике. Функция оставлена из предыдущего демонстрационного варианта 3.

• Ограничения: результаты относятся к одной функции, двум вероятностям мутации, одному бюджету и двадцати seed на метод. Другие размерности, параметры, тестовые задачи или сравнение по времени здесь не исследовались.

Приложение А. Полный листинг main.py

Ниже приведён исходный код, который был запущен для получения показанных результатов. Числовые данные и графики сохранены в каталоге results/ архива.

```
#!/usr/bin/env python3
"""
Лабораторная работа № 1.
Многомерная оптимизация сложной функции генетическим алгоритмом.

Функция Растригина (строка V=3 методички); индивидуально заданная размерность d=5+(22 mod 6)=9.
Ключевые операторы генетического алгоритма реализованы самостоятельно.
"""

from __future__ import annotations

import argparse
import csv
import json
from dataclasses import dataclass
from pathlib import Path
from typing import Iterable

import matplotlib.pyplot as plt
import numpy as np


@dataclass(frozen=True)
class GAConfig:
    dimension: int = 9
    lower_bound: float = -5.12
    upper_bound: float = 5.12
    population_size: int = 80
    generations: int = 250
    tournament_size: int = 3
    crossover_probability: float = 0.90
    blx_alpha: float = 0.50
    mutation_probability: float = 1.0 / 18.0
    mutation_sigma_start: float = 2.50
    mutation_sigma_end: float = 0.03
    elite_count: int = 2

    @property
    def evaluation_budget(self) -> int:
        # Первое поколение оценивается целиком, затем оцениваются только новые потомки.
        return self.population_size + (
            self.generations - 1
        ) * (self.population_size - self.elite_count)


def rastrigin(population: np.ndarray) -> np.ndarray:
    """Векторизованная функция Растригина для массива shape=(n, d)."""
    population = np.asarray(population, dtype=float)
    if population.ndim == 1:
        population = population.reshape(1, -1)
    d = population.shape[1]
    return 10.0 * d + np.sum(
        population**2 - 10.0 * np.cos(2.0 * np.pi * population),
        axis=1,
    )


def initialize_population(config: GAConfig, rng: np.random.Generator) -> np.ndarray:
    return rng.uniform(
        config.lower_bound,
        config.upper_bound,
        size=(config.population_size, config.dimension),
    )


def tournament_indices(
    fitness: np.ndarray,
    count: int,
    tournament_size: int,
    rng: np.random.Generator,
) -> np.ndarray:
    """Индексы победителей независимых турниров (задача минимизации)."""
    candidates = rng.integers(
        0,
        len(fitness),
        size=(count, tournament_size),
    )
```

```
    candidate_fitness = fitness[candidates]
    winner_positions = np.argmin(candidate_fitness, axis=1)
    return candidates[np.arange(count), winner_positions]


def blx_alpha_crossover(
    parent_a: np.ndarray,
    parent_b: np.ndarray,
    alpha: float,
    rng: np.random.Generator,
) -> np.ndarray:
    """BLX-alpha для двух наборов вещественных родителей."""
    low_parent = np.minimum(parent_a, parent_b)
    high_parent = np.maximum(parent_a, parent_b)
    interval = high_parent - low_parent
    low = low_parent - alpha * interval
    high = high_parent + alpha * interval
    return rng.uniform(low, high)


def gaussian_mutation(
    children: np.ndarray,
    probability: float,
    sigma: float,
    rng: np.random.Generator,
) -> np.ndarray:
    """Независимая гауссовская мутация каждой координаты."""
    mask = rng.random(children.shape) < probability
    noise = rng.normal(0.0, sigma, size=children.shape)
    return children + mask * noise


def repair_bounds(children: np.ndarray, config: GAConfig) -> np.ndarray:
    """Ремонт ограничений: отсечение координат по границам области."""
    return np.clip(children, config.lower_bound, config.upper_bound)


def mutation_sigma(config: GAConfig, generation: int) -> float:
    """Экспоненциальное уменьшение масштаба мутации."""
    if config.generations <= 1:
        return config.mutation_sigma_end
    progress = generation / (config.generations - 1)
    ratio = config.mutation_sigma_end / config.mutation_sigma_start
    return config.mutation_sigma_start * (ratio**progress)


def run_ga(seed: int, config: GAConfig) -> dict:
    rng = np.random.default_rng(seed)
    population = initialize_population(config, rng)
    fitness = rastrigin(population)
    evaluations = config.population_size

    best_index = int(np.argmin(fitness))
    best_value = float(fitness[best_index])
    best_vector = population[best_index].copy()
    trajectory = [best_value]

    child_count = config.population_size - config.elite_count
    for generation in range(1, config.generations):
        elite_indices = np.argsort(fitness)[: config.elite_count]
        elites = population[elite_indices].copy()
        elite_fitness = fitness[elite_indices].copy()

        parent_indices = tournament_indices(
            fitness,
            count=2 * child_count,
            tournament_size=config.tournament_size,
            rng=rng,
        )
        parent_a = population[parent_indices[:child_count]]
        parent_b = population[parent_indices[child_count:]]

        do_crossover = rng.random(child_count) < config.crossover_probability
        children = blx_alpha_crossover(
            parent_a,
            parent_b,
            config.blx_alpha,
            rng,
        )
        children[~do_crossover] = parent_a[~do_crossover]

        sigma = mutation_sigma(config, generation)
        children = gaussian_mutation(
            children,
```

```
            config.mutation_probability,
            sigma,
            rng,
        )
        children = repair_bounds(children, config)

        child_fitness = rastrigin(children)
        evaluations += child_count
        population = np.vstack((elites, children))
        fitness = np.concatenate((elite_fitness, child_fitness))

        generation_best_index = int(np.argmin(fitness))
        generation_best_value = float(fitness[generation_best_index])
        if generation_best_value < best_value:
            best_value = generation_best_value
            best_vector = population[generation_best_index].copy()

        trajectory.append(best_value)

    return {
        "seed": seed,
        "algorithm": "GA",
        "final_best": best_value,
        "best_vector": best_vector,
        "trajectory": np.asarray(trajectory, dtype=float),
        "evaluations": evaluations,
    }


def run_random_search(seed: int, config: GAConfig) -> dict:
    """
    Случайный поиск с тем же бюджетом вычислений функции.
    Для сопоставимости траектория содержит столько же точек, сколько поколений ГА.
    """
    rng = np.random.default_rng(seed)
    initial = rng.uniform(
        config.lower_bound,
        config.upper_bound,
        size=(config.population_size, config.dimension),
    )
    values = rastrigin(initial)
    evaluations = config.population_size

    best_index = int(np.argmin(values))
    best_value = float(values[best_index])
    best_vector = initial[best_index].copy()
    trajectory = [best_value]

    batch = config.population_size - config.elite_count
    for _ in range(1, config.generations):
        points = rng.uniform(
            config.lower_bound,
            config.upper_bound,
            size=(batch, config.dimension),
        )
        values = rastrigin(points)
        evaluations += batch
        index = int(np.argmin(values))
        if float(values[index]) < best_value:
            best_value = float(values[index])
            best_vector = points[index].copy()
        trajectory.append(best_value)

    return {
        "seed": seed,
        "algorithm": "Random search",
        "final_best": best_value,
        "best_vector": best_vector,
        "trajectory": np.asarray(trajectory, dtype=float),
        "evaluations": evaluations,
    }


def summary(values: Iterable[float]) -> dict:
    array = np.asarray(list(values), dtype=float)
    return {
        "best": float(np.min(array)),
        "mean": float(np.mean(array)),
        "median": float(np.median(array)),
        "std": float(np.std(array, ddof=1)),
        "worst": float(np.max(array)),
    }
def save_runs_csv(path: Path, labeled_runs: list[tuple[str, dict]]) -> None:
    with path.open("w", newline="", encoding="utf-8") as file:
```

```
        writer = csv.writer(file)
        header = [
            "configuration",
            "algorithm",
            "seed",
            "evaluations",
            "final_best",
        ] + [f"x{i}" for i in range(1, labeled_runs[0][1]["best_vector"].size + 1)]
        writer.writerow(header)
        for label, run in labeled_runs:
            writer.writerow(
                [
                    label,
                    run["algorithm"],
                    run["seed"],
                    run["evaluations"],
                    f'{run["final_best"]:.15g}',
                    *[f"{x:.15g}" for x in run["best_vector"]],
                ]
            )


def save_trajectories_csv(path: Path, labeled_runs: list[tuple[str, dict]]) -> None:
    with path.open("w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow(["configuration", "seed", "generation", "best_so_far"])
        for label, run in labeled_runs:
            for generation, value in enumerate(run["trajectory"]):
                writer.writerow(
                    [label, run["seed"], generation, f"{value:.15g}"]
                )


def plot_primary_convergence(runs: list[dict], path: Path) -> None:
    trajectories = np.vstack([run["trajectory"] for run in runs])
    minimum = np.min(trajectories, axis=0)
    mean = np.mean(trajectories, axis=0)
    maximum = np.max(trajectories, axis=0)
    generations = np.arange(trajectories.shape[1])

    plt.figure(figsize=(8.2, 4.8))
    plt.plot(generations, np.maximum(minimum, 1e-12), label="Минимум по 20 запускам")
    plt.plot(generations, np.maximum(mean, 1e-12), label="Среднее по 20 запускам")
    plt.plot(generations, np.maximum(maximum, 1e-12), label="Максимум по 20 запускам")
    plt.yscale("log")
    plt.xlabel("Поколение")
    plt.ylabel("Лучшее f(x) на текущий момент, лог. шкала")
    plt.title("Сходимость ГА: функция Растригина, d=9")
    plt.legend()
    plt.tight_layout()
    plt.savefig(path, dpi=180)
    plt.close()


def plot_mean_comparison(
    low_mutation_runs: list[dict],
    high_mutation_runs: list[dict],
    random_runs: list[dict],
    path: Path,
) -> None:
    generations = np.arange(len(low_mutation_runs[0]["trajectory"]))
    low_mean = np.mean(
        np.vstack([run["trajectory"] for run in low_mutation_runs]),
        axis=0,
    )
    high_mean = np.mean(
        np.vstack([run["trajectory"] for run in high_mutation_runs]),
        axis=0,
    )
    random_mean = np.mean(
        np.vstack([run["trajectory"] for run in random_runs]),
        axis=0,
    )

    plt.figure(figsize=(8.2, 4.8))
    plt.plot(generations, np.maximum(low_mean, 1e-12), label="ГА: p_mut = 1/(2d)")
    plt.plot(generations, np.maximum(high_mean, 1e-12), label="ГА: p_mut = 2/d")
    plt.plot(generations, np.maximum(random_mean, 1e-12), label="Случайный поиск")
    plt.yscale("log")
    plt.xlabel("Поколение / эквивалентный пакет оценок")
    plt.ylabel("Среднее лучшее f(x), лог. шкала")
    plt.title("Сравнение конфигураций и случайного поиска")
    plt.legend()
    plt.tight_layout()
```

```
    plt.savefig(path, dpi=180)
    plt.close()


def plot_final_boxplot(
    low_mutation_runs: list[dict],
    high_mutation_runs: list[dict],
    random_runs: list[dict],
    path: Path,
) -> None:
    data = [
        [run["final_best"] for run in low_mutation_runs],
        [run["final_best"] for run in high_mutation_runs],
        [run["final_best"] for run in random_runs],
    ]
    plt.figure(figsize=(7.8, 4.8))
    plt.boxplot(data, tick_labels=["ГА: p=1/(2d)", "ГА: p=2/d", "Случайный поиск"])
    plt.ylabel("Итоговое лучшее значение f(x)")
    plt.title("Распределение результатов 20 независимых запусков")
    plt.tight_layout()
    plt.savefig(path, dpi=180)
    plt.close()


def plot_flowchart(path: Path) -> None:
    stages = [
        (0.50, 0.92, "Инициализация\nпопуляции"),
        (0.50, 0.79, "Оценка f(x)"),
        (0.50, 0.66, "Турнирный отбор\nродителей"),
        (0.50, 0.53, "BLX-α кроссовер"),
        (0.50, 0.40, "Гауссовская мутация\n+ repair границ"),
        (0.50, 0.27, "Элитизм и формирование\nнового поколения"),
        (0.50, 0.14, "Критерий останова?"),
    ]

    plt.figure(figsize=(5.8, 8.0))
    ax = plt.gca()
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.axis("off")
    for x, y, label in stages:
        ax.text(
            x,
            y,
            label,
            ha="center",
            va="center",
            bbox=dict(boxstyle="round,pad=0.45"),
        )

    for i in range(len(stages) - 1):
        ax.annotate(
            "",
            xy=(stages[i + 1][0], stages[i + 1][1] + 0.045),
            xytext=(stages[i][0], stages[i][1] - 0.045),
            arrowprops=dict(arrowstyle="->"),
        )

    ax.annotate(
        "нет",
        xy=(0.40, 0.79),
        xytext=(0.12, 0.25),
        ha="center",
        va="center",
        arrowprops=dict(
            arrowstyle="->",
            connectionstyle="arc3,rad=-0.35",
        ),
    )
    ax.annotate(
        "да",
        xy=(0.50, 0.045),
        xytext=(0.50, 0.095),
        ha="center",
        va="center",
        arrowprops=dict(arrowstyle="->"),
    )
    ax.text(
        0.50,
        0.025,
        "Возврат best-ever",
        ha="center",
        va="center",
        bbox=dict(boxstyle="round,pad=0.35"),
```

```
    )

    plt.title("Схема генетического алгоритма")
    plt.tight_layout()
    plt.savefig(path, dpi=180)
    plt.close()


def run_experiment(output_dir: Path, runs_count: int, seed_base: int) -> dict:
    output_dir.mkdir(parents=True, exist_ok=True)

    config_low = GAConfig(mutation_probability=1.0 / (2.0 * 9.0))
    config_high = GAConfig(mutation_probability=2.0 / 9.0)

    seeds = [seed_base + i for i in range(runs_count)]

    low_runs = [run_ga(seed, config_low) for seed in seeds]
    high_runs = [run_ga(seed, config_high) for seed in seeds]
    random_runs = [run_random_search(seed, config_low) for seed in seeds]

    labeled_runs: list[tuple[str, dict]] = []
    labeled_runs.extend(("GA_low_mutation", run) for run in low_runs)
    labeled_runs.extend(("GA_high_mutation", run) for run in high_runs)
    labeled_runs.extend(("random_search", run) for run in random_runs)
    save_runs_csv(output_dir / "runs.csv", labeled_runs)
    save_trajectories_csv(output_dir / "trajectories.csv", labeled_runs)

    summaries = {
        "variant": "Rastrigin (method row 3); user-specified dimension 9",
        "function": "Rastrigin",
        "dimension": config_low.dimension,
        "theoretical_minimum": 0.0,
        "evaluation_budget": config_low.evaluation_budget,
        "runs_per_configuration": runs_count,
        "seed_base": seed_base,
        "GA_low_mutation": {
            "mutation_probability": config_low.mutation_probability,
            **summary(run["final_best"] for run in low_runs),
        },
        "GA_high_mutation": {
            "mutation_probability": config_high.mutation_probability,
            **summary(run["final_best"] for run in high_runs),
        },
        "random_search": summary(run["final_best"] for run in random_runs),
    }

    best_run = min(low_runs, key=lambda run: run["final_best"])
    summaries["best_run"] = {
        "seed": best_run["seed"],
        "value": best_run["final_best"],
        "vector": [float(x) for x in best_run["best_vector"]],
    }

    with (output_dir / "summary.json").open("w", encoding="utf-8") as file:
        json.dump(summaries, file, ensure_ascii=False, indent=2)

    parameters = {
        "variant": "Rastrigin (method row 3); user-specified dimension 9",
        "dimension_formula": "d = 5 + (22 mod 6) = 9; the number 22 is NOT a formal V in the methods (V=1..20)",
        "variant_note": "Rastrigin is from row V=3; dimension 9 was separately requested",
        "dimension": config_low.dimension,
        "bounds": [config_low.lower_bound, config_low.upper_bound],
        "population_size": config_low.population_size,
        "generations": config_low.generations,
        "evaluation_budget": config_low.evaluation_budget,
        "tournament_size": config_low.tournament_size,
        "crossover": "BLX-alpha",
        "crossover_probability": config_low.crossover_probability,
        "blx_alpha": config_low.blx_alpha,
        "mutation": "Gaussian, per coordinate",
        "mutation_probability_primary": config_low.mutation_probability,
        "mutation_probability_comparison": config_high.mutation_probability,
        "mutation_sigma_start": config_low.mutation_sigma_start,
        "mutation_sigma_end": config_low.mutation_sigma_end,
        "mutation_sigma_schedule": "exponential decay",
        "bounds_handling": "clip",
        "elitism": config_low.elite_count,
        "selection": "tournament, k=3",
        "independent_runs": runs_count,
        "seed_base": seed_base,
    }
    with (output_dir / "parameters.json").open("w", encoding="utf-8") as file:
        json.dump(parameters, file, ensure_ascii=False, indent=2)
    plot_primary_convergence(low_runs, output_dir / "convergence_primary.png")
```

```
    plot_mean_comparison(
        low_runs,
        high_runs,
        random_runs,
        output_dir / "comparison_mean.png",
    )
    plot_final_boxplot(
        low_runs,
        high_runs,
        random_runs,
        output_dir / "final_boxplot.png",
    )
    plot_flowchart(output_dir / "flowchart.png")

    return summaries


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--output-dir",
        type=Path,
        default=Path("results"),
        help="Каталог результатов эксперимента.",
    )
    parser.add_argument("--runs", type=int, default=20)
    parser.add_argument("--seed-base", type=int, default=2026100200)
    return parser.parse_args()


def main() -> None:
    args = parse_args()
    summaries = run_experiment(args.output_dir, args.runs, args.seed_base)
    print(json.dumps(summaries, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```

Приложение Б. Тесты и воспроизводимость

Полный листинг test_lab1.py. Синтаксис и результаты пяти тестов проверены запуском на программном комплексе, включённом в архив.

```
import unittest
import numpy as np

from main import GAConfig, gaussian_mutation, rastrigin, repair_bounds, run_ga


class Lab1Tests(unittest.TestCase):
    def test_rastrigin_global_minimum(self):
        value = rastrigin(np.zeros((1, 9)))[0]
        self.assertAlmostEqual(float(value), 0.0, places=12)

    def test_repair_bounds(self):
        config = GAConfig()
        values = np.array([[100.0] * 9, [-100.0] * 9])
        repaired = repair_bounds(values, config)
        self.assertTrue(np.all(repaired <= config.upper_bound))
        self.assertTrue(np.all(repaired >= config.lower_bound))

    def test_reproducibility(self):
        config = GAConfig(generations=20)
        first = run_ga(12345, config)
        second = run_ga(12345, config)
        self.assertAlmostEqual(first["final_best"], second["final_best"], places=12)
        self.assertTrue(np.allclose(first["best_vector"], second["best_vector"]))

    def test_dimension(self):
        config = GAConfig()
        result = run_ga(31415, config)
        self.assertEqual(len(result["best_vector"]), 9)

    def test_budget(self):
        config = GAConfig(generations=20)
        result = run_ga(1, config)
        self.assertEqual(result["evaluations"], config.evaluation_budget)


if __name__ == "__main__":
    unittest.main()
```

Структура машиночитаемых файлов: results/parameters.json содержит все параметры; results/summary.json — статистику и лучший вектор; results/runs.csv — 60 результатов; results/trajectories.csv — 15 000 точек траекторий; PNG-файлы — исходные графики.